CS
v2

CreditSense v2

AI-Powered Loan Approval Agent

Deterministic Policy Engine · RAG · LangGraph · Bilingual Reports

Streamlit | FastAPI | LangGraph | ChromaDB | Groq LLaMA-3.3-70b | ONNX MiniLM

PROJECT TEAM

Mitul Bhatia — Project Lead & Full-Stack Developer

Vaibhav — Technical Documentation & Report Strategy

Sarvjeet — Media, Presentation & Demo Lead

Hardik — Product Ideation & Business Strategy

. Table of Contents

Abstract	3
1. Introduction	3
2. Team Contributions	4
3. System Architecture	5
4. LangGraph Orchestration	6
5. RAG Pipeline	7
6. Data-Flow Sequence	8
7. Policy & Scoring Engine	8
8. Financial Metrics	9
9. Agent State Architecture	10
10. Implementation & Methodology	10
11. API Reference	11
12. Deployment	12
13. Functional Requirements Status	13
14. Gaps & Future Roadmap	14
15. Conclusion	15

. Abstract

The traditional loan approval process is hindered by manual document verification, subjective risk evaluation, and bottlenecks in interpreting complex financial regulations. This report presents **CreditSense v2**, an intelligent, semi-autonomous credit risk assessment platform built using a hybrid architecture comprising **Streamlit**, **FastAPI**, and a **LangGraph-orchestrated LLM pipeline**. The platform conducts interactive borrower intake, queries regulatory documents (RBI and NBFC guidelines) via Retrieval-Augmented Generation (RAG) using ONNX-accelerated embeddings, and calculates an explicit risk score using a deterministic policy engine. The output is a bilingual (English and Hindi), highly explainable financial report that empowers loan officers to make rapid, compliant, and auditable credit decisions. The system is fully deployable on free-tier cloud infrastructure by replacing the 2GB+ PyTorch/SentenceTransformers stack with Chroma's native **ONNXMiniLM_L6_V2** engine at under 50 MB.

1. Introduction

In modern financing, decision latency directly impacts customer acquisition and operational overhead. However, completely offloading credit decisions to AI models introduces severe non-compliance risks — Large Language Models (LLMs) frequently hallucinate rules or invent financial metrics when left unconstrained.

CreditSense v2 solves this by implementing a **deterministic constraint envelope** over the AI. The LLM is used strictly for conversational data extraction, summarization, and translation, while the actual underwriting decision (Approve, Conditional, or Escalate) is grounded entirely in mathematics and fixed policy matrices. This design keeps the system both intelligent and auditable.

1.1 Core Deliverables

- **Conversational UI** — Dynamic Streamlit frontend for bilingual borrower intake and interactive Q&A.;
- **LangGraph Orchestrator** — Strict 6-node graph: Guardrail → Conversation → RAG → Policy → Report → Translate.
- **Lightweight RAG** — ChromaDB + ONNXMiniLM_L6_V2 (50 MB) replacing 2 GB+ PyTorch stack.
- **Deterministic Policy Engine** — Heuristic risk scorer + rule-based underwriting checks with explicit score computation.
- **Bilingual PDF Export** — fpdf2 + Noto Devanagari font for formatted English/Hindi downloadable reports.
- **REST API** — FastAPI backend exposing stateless endpoints for all agent operations.

1.2 Milestone 2 Focus Areas

Milestone 2 graduates the system from a terminal prototype (Milestone 1) to a fully cloud-deployable, production-shaped dual-stack application. Key upgrades include the migration to ONNX embeddings,

the LangGraph deterministic orchestration layer, bilingual PDF generation, and deployment manifests for Streamlit Cloud and Render.

2. Team Contributions & Roles

Development and delivery of this robust platform was achieved through a structured division of labour ensuring specific expertise was applied at every layer of the product lifecycle.

Member	Role	Responsibilities
Mitul Bhatia	Project Lead & Full-Stack Developer	Spearheaded the overall software architecture. Engineered the FastAPI backend, Streamlit frontend, LangGraph orchestration, local RAG integration with ONNX embeddings, and cloud deployment pipelines.
Vaibhav	Technical Documentation & Report Strategy	Structured and developed the comprehensive project documentation, system architecture records, READMEs, and the formal technical submission report with all API and architecture deep-dives.
Sarvjeet	Media & Presentation Lead	Designed and delivered the final project presentation, mapped the technical flow into digestible visual slides, and produced the final demo walkthrough video for milestone evaluation.
Hardik	Product Ideation & Business Strategy	Led initial concept framing, conceptualised RBI guidelines integration, structured the business logic requirements (DTI, LTV ratios), and defined the target user experience for loan officers.

3. System Architecture & Design

CreditSense v2 utilises a modern multi-tier microservice architecture ensuring speed, auditability, and scalability. The platform is divided into five distinct layers that communicate via well-defined interfaces.

3.1 Full System Architecture Diagram

The diagram below shows the exact runtime layout of CreditSense v2 — all five layers, all six LangGraph nodes, external services, and data-flow arrows:

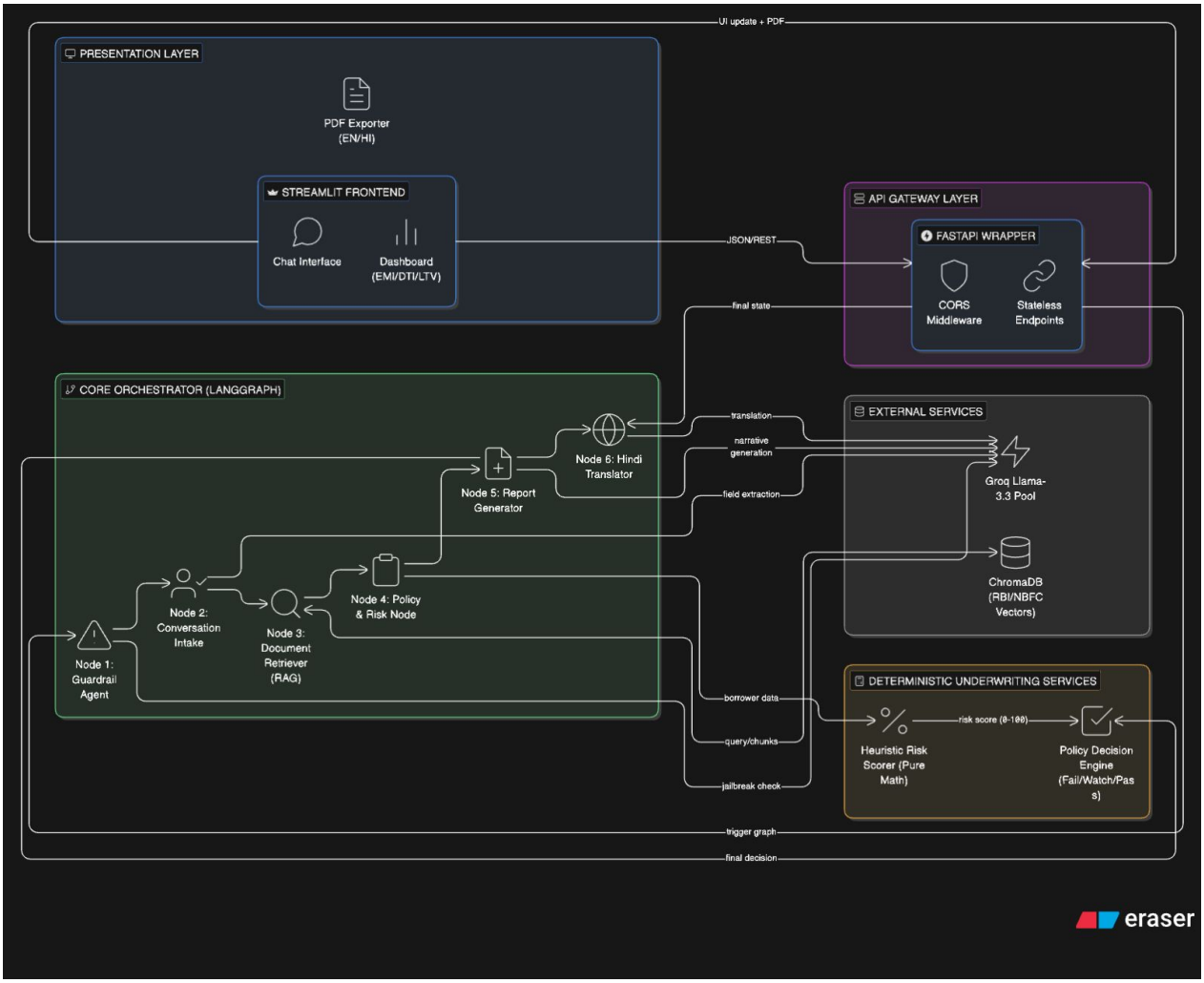

Figure 3.1 — CreditSense v2 Complete System Architecture (all layers & data-flow)

3.2 Layer Responsibilities

Layer	Technology	Key Responsibilities
Presentation	Streamlit Cloud	Borrower intake form, chat interface, EMI/DTI/LTV dashboard, PDF download buttons

API Gateway	FastAPI + CORS	Stateless REST endpoints, state normalisation, graph invocation, ingestion trigger
Orchestrator	LangGraph	6-node deterministic graph: guardrail → conversation → RAG → policy → report → translate
External Services	Groq + ChromaDB	LLM prompting (Llama-3.3-70b), ONNX vector store for RBI/NBFC regulatory corpus
Underwriting	Pure Python Math	Heuristic risk scorer (0-100), rule-based policy engine → APPROVE / CONDITIONAL / ESCALATE

3.3 ONNX Migration — Why It Matters

A critical requirement for Milestone 2 was ensuring cloud deployability on free-tier infrastructure (Streamlit Community Cloud, Render Free). The original RAG pipeline used **sentence_transformers + PyTorch** which forced a 2 GB+ dependency, causing memory exhaustion and deployment crashes. The solution was migrating to ChromaDB's native **ONNXMiniLM_L6_V2** engine:

Metric	PyTorch Stack (Before)	ONNX Stack (After)
Embedding Model	sentence-transformers	ONNXMiniLM_L6_V2
Memory Footprint	2.2 GB+	~50 MB ✓
Vector Dimensions	384-dim	384-dim (identical)
Inference Speed	Baseline	~5% of PyTorch load time
Free-tier deploy	■ Crashes on OOM	■ Stable

4. LangGraph Orchestration

LangGraph's **StateGraph** is the core innovation of Milestone 2. It prevents the LLM from leaping directly to a generated report before mandatory calculations occur, ensuring deterministic execution order regardless of user input.

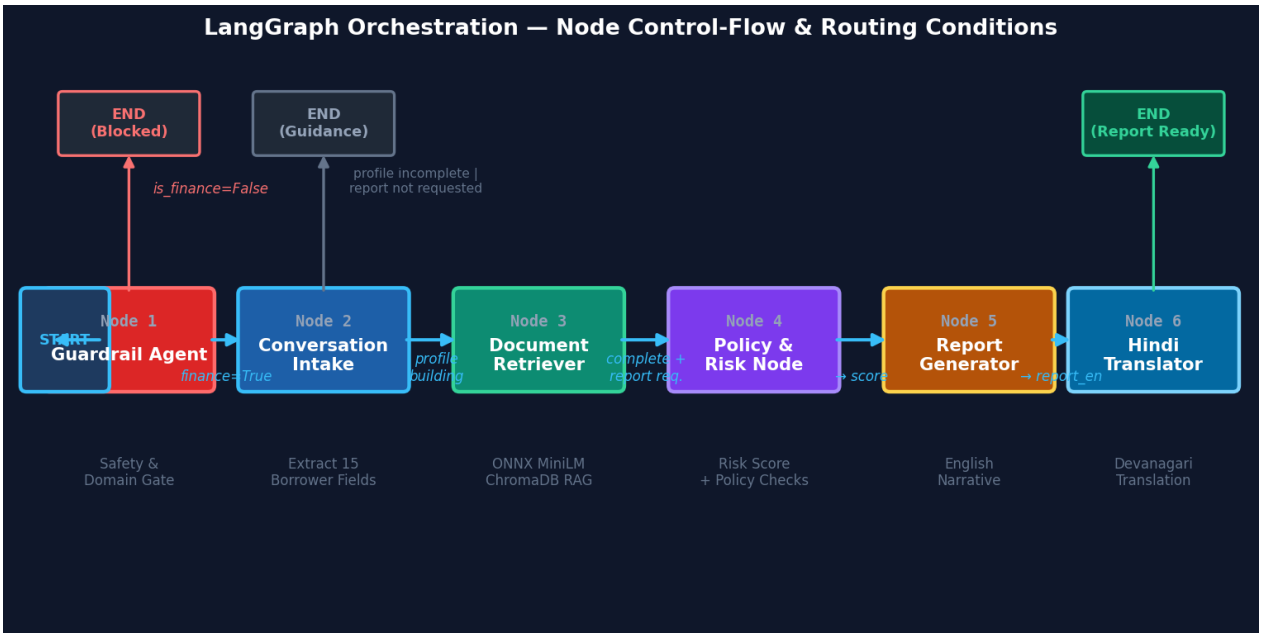

Figure 4.1 — LangGraph 6-Node Control-Flow with Routing Conditions

4.1 Node-by-Node Breakdown

Node	Name	Primary Function	Key Outputs
1	Guardrail Agent	Safety & domain gating — blocks jailbreaks, non-finance queries, abusive content. Uses Groq Llama-3.1-8b-instant + deterministic regex fallback.	is_finance_query guardrail_reason
2	Conversation Intake	Extracts 15 specific borrower fields from natural language. Uses regex first, then LLM augmentation. Computes DTI/EMI/LTV immediately after each update.	collected, missing_fields profile_complete
3	Document Retriever	Builds profile-aware query, retrieves top-8 chunks from ChromaDB using ONNX MiniLM similarity search against the RBI/NBFC corpus.	rag_chunks citations

4	Policy & Risk Node	Runs heuristic risk scorer (pure math) + 6 deterministic policy checks. Aggregates FAIL/WATCH penalties from base score 100.	policy_checks, risk_flags decision, decision_score
5	Report Generator	Transcribes math constraints and policy checks into a structured English credit narrative. Uses Llama-3.3-70b with deterministic template fallback.	report_en assistant_reply
6	Hindi Translator	Translates report_en to Hindi using Llama-3.3-70b with Devanagari validation. Falls back to deterministic keyword mapper if LLM fails.	report_hi

4.2 Routing Conditions

- Node 1 → END: **is_finance_query == False** (blocked / off-topic)
- Node 1 → Node 2: **is_finance_query == True**
- Node 2 → END: **profile_complete == False** OR **report_requested == False**
- Node 2 → Node 3: **profile_complete == True AND report_requested == True**
- Nodes 3 → 4 → 5 → 6 → END: *deterministic linear chain — no further branching*

5. RAG Pipeline — Retrieval-Augmented Generation

The RAG pipeline grounds every loan guidance response and final report in actual regulatory text retrieved from the project's RBI/NBFC corpus. It consists of two distinct phases: **Ingestion** and **Retrieval**.

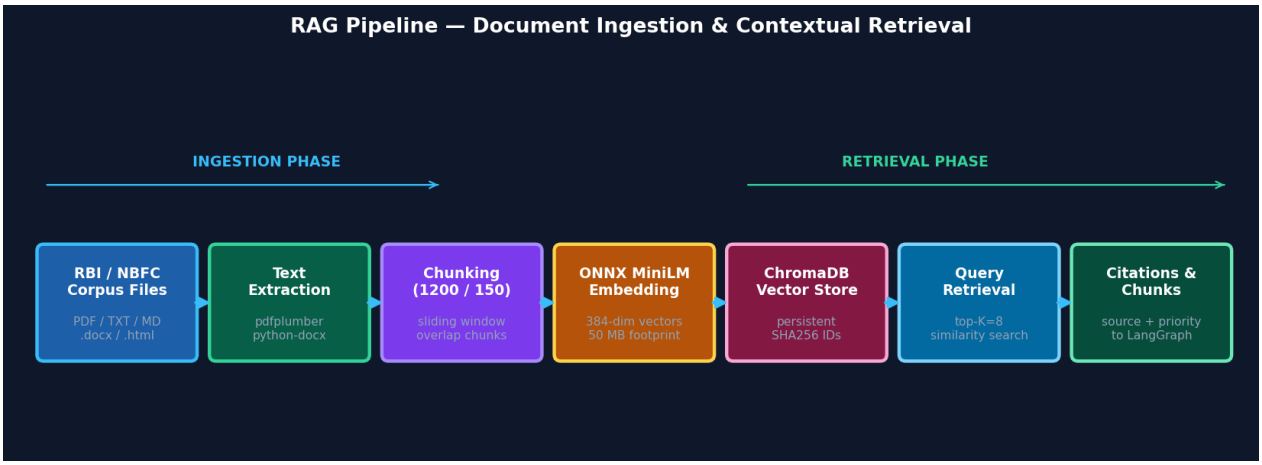

Figure 5.1 — RAG Pipeline: Ingestion Phase (left) and Retrieval Phase (right)

5.1 Ingestion Phase

The ingestion script (scripts/ingest.py) recursively scans the source directory for supported file types (.pdf, .txt, .md, .docx, .html). Text is extracted per-format, then chunked using a sliding-window strategy:

Step	Parameter / Tool	Detail
File Discovery	iter_docs()	Recursive scan; excludes hidden files, .DS_Store
Text Extraction	pdfplumber / python-docx	Per-format extractors; UTF-8 fallback for text files
Chunking	chunk_size=1200, overlap=150	Sliding window with overlap for context continuity
Embedding	ONNXMiniLM_L6_V2()	384-dim vectors; memoised with lru_cache(1)
Upsert to Chroma	SHA256 chunk IDs	Metadata: source_file, priority (P0/P1/P2), chunk_index

5.2 Priority Tagging

Documents are tagged with a priority level during ingestion to weight retrieval relevance:

Priority	Keywords	Example Documents

P0 — Critical	irac, fair, digital, nbfc	NBFC Fair Practices, Digital Lending Guidelines
P1 — High	master, rbi, basel, prudential	RBI Master Circulars, Basel Prudential Norms
P2 — Default	(all other files)	Supplementary reference documents

5.3 Retrieval Phase

At report-generation time, the RAG Retriever node builds a profile-aware query combining: loan purpose, amount, credit score, post-loan DTI, employment type, collateral type, conversation context, and the anchor phrase *'RBI NBFC underwriting fair practices'*. This query is encoded with ONNXMiniLM and matched against ChromaDB using cosine similarity to retrieve the top-8 most relevant regulatory chunks, which are passed as citations to the Policy and Report nodes.

6. Data-Flow Sequence

The platform operates in two distinct runtime paths depending on whether the borrower profile is complete and a report has been requested:

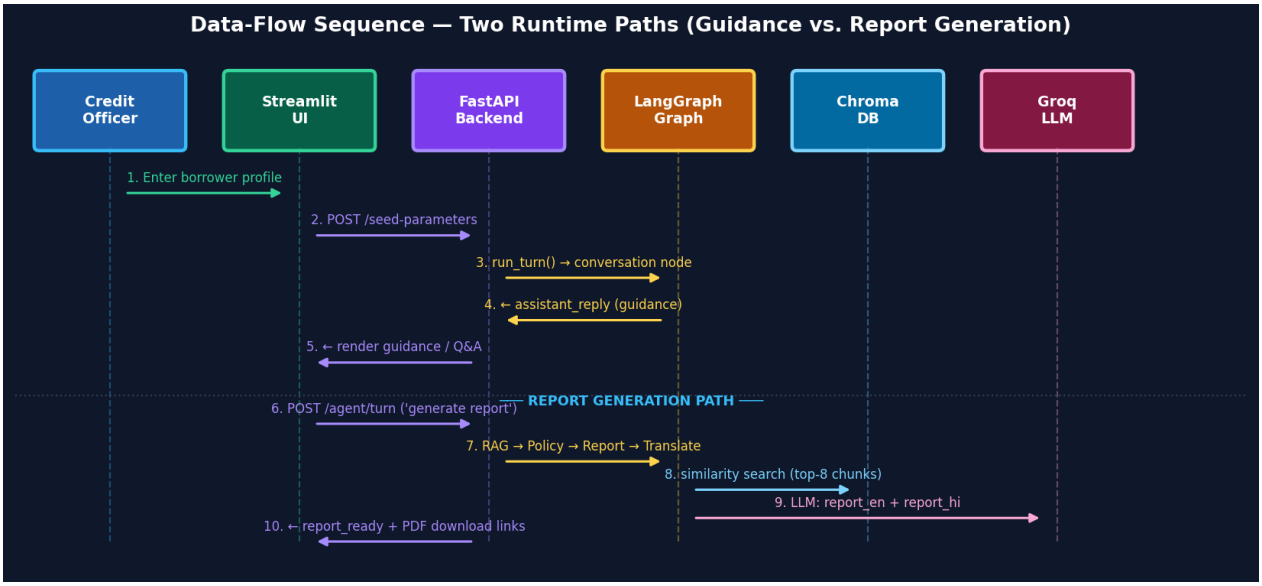

Figure 6.1 — Sequence Diagram: Guidance Path (top) and Report Generation Path (bottom)

6.1 Path A — Guidance Mode

- Credit Officer fills in borrower fields in the Streamlit sidebar.
- UI sends POST /api/v1/agent/seed-parameters with JSON parameters.
- Backend normalises inputs and merges into agent state; missing fields are identified.
- LangGraph runs: Guardrail → Conversation (extracts/updates + computes ratios).
- Agent returns guidance reply or next intake question; no report chain triggered.

6.2 Path B — Report Generation Mode

- Officer sends *'generate report'* trigger; profile is already complete.
- UI sends POST /api/v1/agent/turn with report intent.
- LangGraph routes to full chain: RAG Retriever → Policy Node → Report Node → Translate.
- RAG retrieves top-8 regulatory chunks; Policy computes score and decision class.
- Report Node generates English narrative; Translate Node produces Hindi version.
- UI enables PDF download buttons for both English and Hindi reports.

7. Policy & Scoring Engine

The underwriting decision is entirely deterministic — no LLM hallucination can affect the final score or approval class. This is the most critical safety property of the system.

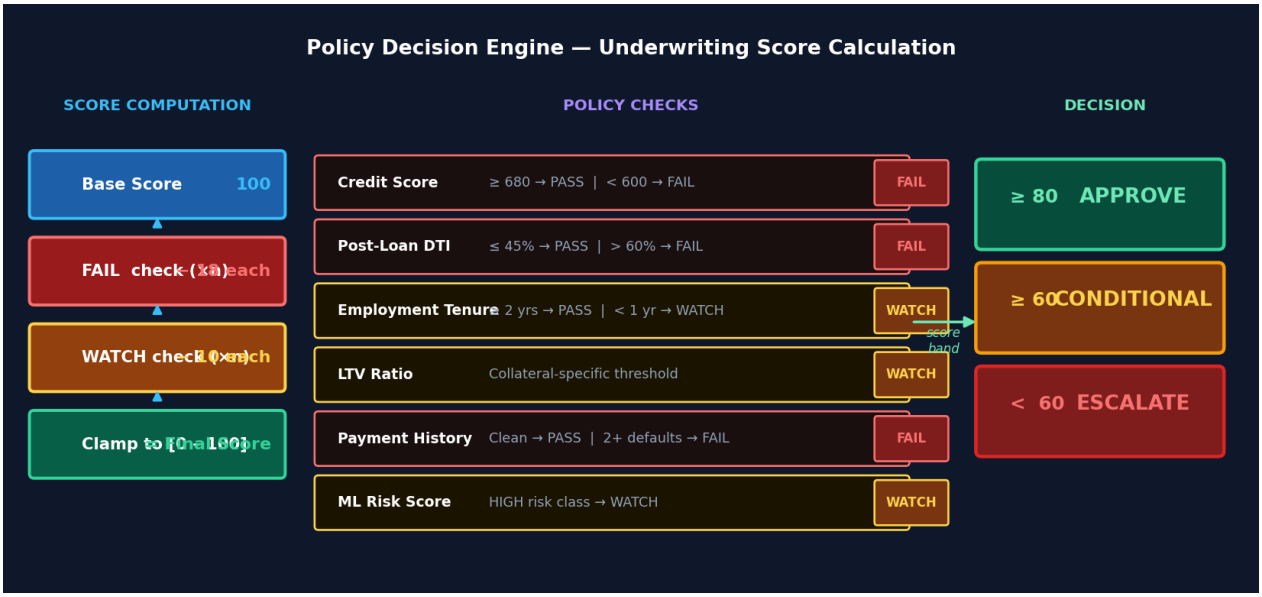

Figure 7.1 — Policy Engine: Score Computation (left), Policy Checks (center), Decision Bands (right)

7.1 Heuristic Risk Scorer (risk_scorer.py)

Score starts at 50 and adjusts by borrower profile factors:

Factor	Condition	Score Adjustment
Credit Score	≥ 750	+25
	≥ 680	+12
	≥ 600	+3
	< 600	–15
Post-Loan DTI	≤ 45%	+18
	≤ 60%	+6
	> 75%	–18
Payment History	Clean	+8
	1 default	–6
	2+ defaults	–14
Employment Years	≥ 2 years	+6

	< 1 year	–4

7.2 Policy Decision Engine (policy_engine.py)

Six checks are applied; each FAIL deducts 18 points, each WATCH deducts 10 points from base 100:

Check	PASS Condition	Flag	Deduction
Credit Score	≥ 680	FAIL	–18
Post-Loan DTI	≤ 45%	FAIL	–18
Employment Tenure	≥ 2 years	WATCH	–10
LTV Ratio	Collateral-type specific	WATCH	–10
Payment History	Clean	FAIL	–18
ML Risk Score	Score ≥ 55 (MEDIUM+)	WATCH	–10

7.3 Decision Bands

Score Range	Decision Class	Meaning
≥ 80 / 100	■ APPROVE	Low risk — loan recommended for approval
60 – 79	■■ CONDITIONAL	Medium risk — conditional with additional review
< 60	■ ESCALATE	High risk — escalate to senior underwriter

8. Financial Metrics Computation

All financial ratios are computed deterministically in services/borrower_metrics.py immediately after each conversational update, ensuring the policy engine always operates on fresh, consistent numbers.

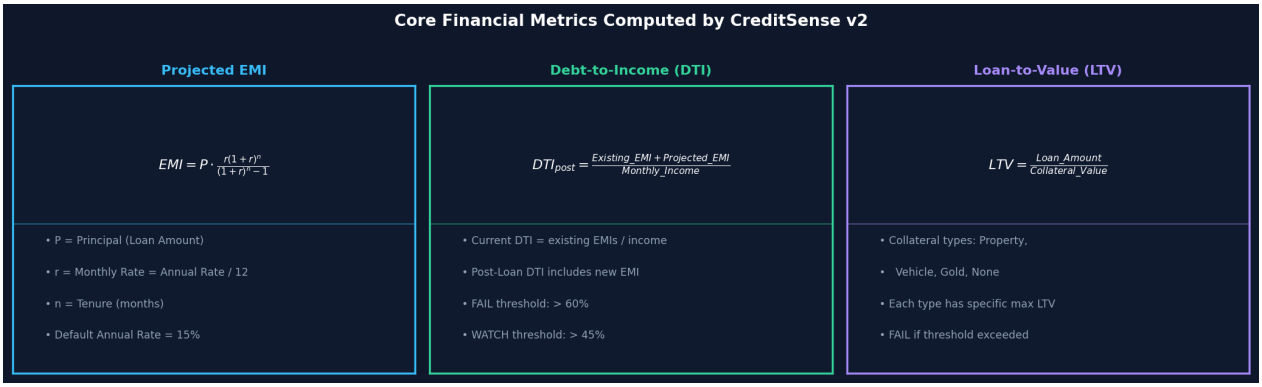

Figure 8.1 — EMI, DTI, and LTV Formulae with Thresholds

Metric	Formula	Thresholds & Notes
Projected EMI	EMI = P × r(1+r)■ / ((1+r)■ – 1) P=Principal, r=monthly rate, n=tenure months	Default annual rate: 15% (configurable via DEFAULT_ANNUAL_INTEREST_RATE)
Current DTI	DTI = Existing Monthly EMI / Monthly Income	Baseline debt burden before new loan
Post-Loan DTI	Post_DTI = (Existing EMI + Projected EMI) / Monthly Income	FAIL if > 60%, WATCH if > 45%, PASS if ≤ 45%
LTV Ratio	LTV = Loan Amount Requested / Collateral Value	Thresholds vary by collateral type: Property / Vehicle / Gold / None

The 15 borrower fields collected during intake include: name, age, employment_type, monthly_income, employment_years, credit_score, existing_loan_count, existing_emi_monthly, payment_history, loan_amount_requested, loan_purpose, loan_tenure_months, collateral_type, collateral_value, and city.

9. Agent State Architecture

The entire application state is a single TypedDict defined in agent/state.py. Since LangGraph has no built-in memory between sessions, the complete state object is passed in every API request and returned with every response, enabling stateless scalability.

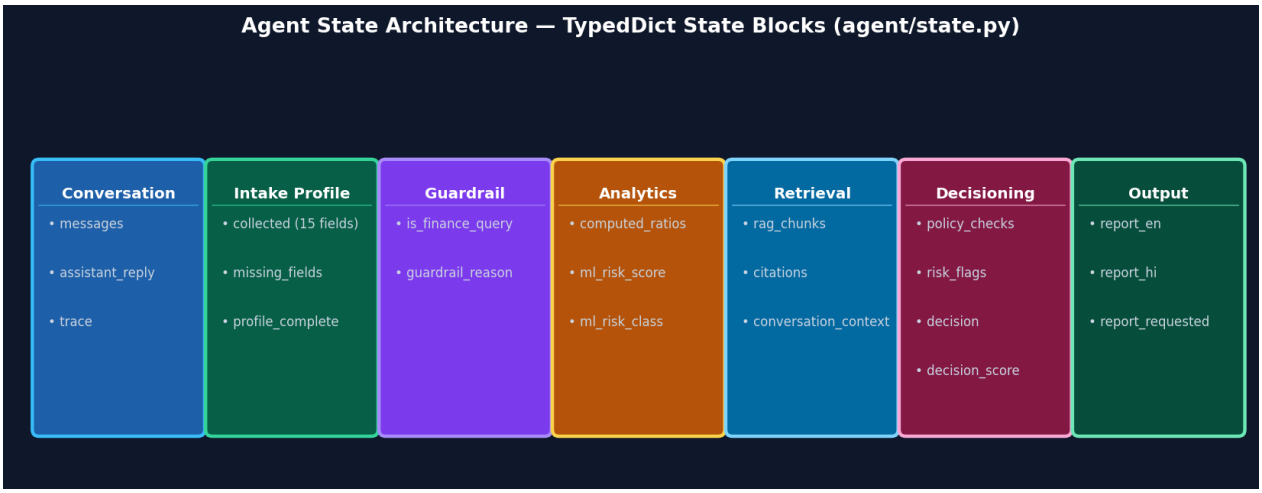

Figure 9.1 — Seven State Blocks comprising the AgentState TypedDict

State Block	Key Fields	Purpose
Conversation	messages, assistant_reply, trace	Conversation history, current reply, execution breadcrumbs
Intake	collected, missing_fields, profile_complete	15-field borrower profile and completion tracking
Guardrail	is_finance_query, guardrail_reason	Domain gating and safety check results
Analytics	computed_ratios, ml_risk_score, ml_risk_class	EMI/DTI/LTV values and heuristic risk signal
Retrieval	rag_chunks, citations, conversation_context	Retrieved regulatory chunks and formatted citations for reports
Decisioning	policy_checks, risk_flags, decision, decision_score	Underwriting checks, flags, final decision class and score
Output	report_en, report_hi, report_requested	Generated English/Hindi reports and PDF readiness flag

10. Implementation & Methodology

10.1 Service Layer Inventory

All business logic is encapsulated in the services/ directory. Each service has a single clearly-defined responsibility:

Service Module	Responsibility
settings.py	Central configuration resolution from environment variables
backend_client.py	Streamlit-to-FastAPI HTTP integration layer
borrower_metrics.py	EMI, current DTI, post-loan DTI, and LTV calculations
groq_pool.py	Thread-safe Groq API key rotation for reliability
groq_client.py	Groq API wrapper with retry logic and error handling
guardrail.py	Safety checks (regex) + finance-domain LLM classification
profile_parser.py	Intake field extraction (regex-first, LLM-augmented)
risk_scorer.py	Heuristic risk score calculation (pure mathematics)
policy_engine.py	Rule-based underwriting checks and score band determination
retriever.py	ONNX MiniLM embedding + ChromaDB similarity search
report_generator.py	English credit assessment narrative generation
translator.py	Hindi translation with Devanagari validation + fallback
pdf_exporter.py	fpdf2 + Noto Devanagari font PDF generation (EN/HI)

10.2 LLM Usage by Node

Node	Model	Fallback
Guardrail	llama-3.1-8b-instant	Deterministic regex keyword classifier
Conversation	llama-3.3-70b-versatile	Deterministic regex field extraction
Report Generation	llama-3.3-70b-versatile	Deterministic report template
Hindi Translation	llama-3.3-70b-versatile	Deterministic Hindi keyword mapper

10.3 Bilingual PDF Export

Financial inclusivity requires accessible language. The system uses the **fpdf2** library with a custom Unicode-compliant **Noto Devanagari** font to output formatted, downloadable PDF records of the complete transaction and regulatory citations. The PDF export fallback chain guarantees output even if the LLM is unavailable: the deterministic report/translation template is used, and the PDF is still generated from that content.

10.4 Reliability Patterns

- **Deterministic fallbacks** at every LLM call point ensure functionality when Groq is unavailable.
- **API key rotation** via groq_pool.py provides thread-safe round-robin across multiple Groq keys.
- **Defensive state merge** in the graph runner prevents key loss across node transitions.
- **Backend/UI decoupling** via HTTP allows independent scaling of frontend and API layers.
- **CORS middleware** configured via BACKEND_CORS_ORIGINS environment variable.

11. API Reference

The FastAPI backend (api.py) exposes five REST endpoints at http://localhost:8010 (local) or the configured Render base URL. The API is stateless — the complete agent state is sent and received with every request.

Method	Endpoint	Purpose
GET	/api/v1/health	Health check + Chroma config metadata
GET	/api/v1/agent/state/initial	Returns fresh empty AgentState for new sessions
POST	/api/v1/agent/turn	Runs one complete LangGraph turn; returns reply + state
POST	/api/v1/agent/seed-parameters	Injects borrower form fields; returns profile completeness
POST	/api/v1/ingest	Triggers corpus ingestion into ChromaDB

11.1 Agent Turn Response Fields

Field	Type	Description
assistant_reply	string	Text response from current agent turn
state	object	Full AgentState — must be sent in next request
report_ready	bool	True when both report_en and report_hi are available
decision	string	APPROVE / CONDITIONAL / ESCALATE
decision_score	int	Final policy score (0–100)
citations_count	int	Number of regulatory citations retrieved
retrieved_count	int	Total RAG chunks retrieved

11.2 Seed Parameters — Borrower Field Schema

The /seed-parameters endpoint accepts all 15 borrower fields in a parameters JSON object. The backend applies normalisation rules: int fields are cast via int(float(value)), blank/invalid values are skipped, and if collateral_type == 'none' then collateral_value is forced to 0.0.

Field	Type	Example
name	string	Rahul Sharma

age	int	32
employment_type	string	Salaried
monthly_income	int	120000
employment_years	float	4.0
credit_score	int	742
existing_loan_count	int	1
existing_emi_monthly	float	18000.0
payment_history	string	Clean
loan_amount_requested	float	650000.0
loan_purpose	string	Business
loan_tenure_months	int	60
collateral_type	string	Property
collateral_value	float	1000000.0
city	string	Pune

Error Behaviour: Invalid/missing source directory → HTTP 400. Graph execution failure → HTTP 500. Ingestion failure → HTTP 500.

12. Deployment Architecture

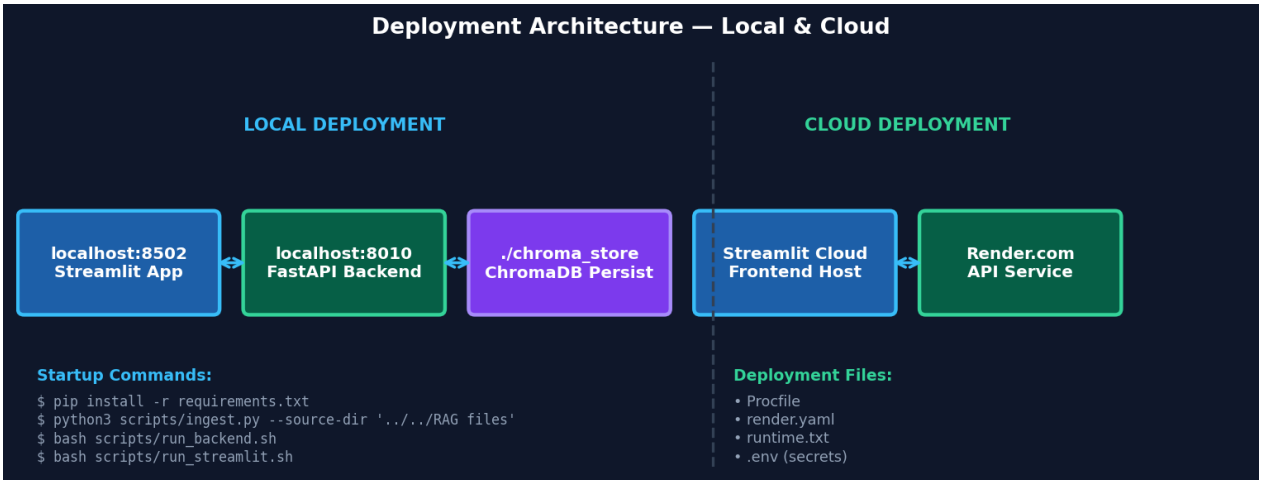

Figure 12.1 — Local and Cloud Deployment Topology

12.1 Local Deployment

The full dual-stack can be run locally from the MILESTONE_2/creditsense/ directory using four commands in sequence:

```
$ pip install -r requirements.txt
$ python3 scripts/ingest.py --source-dir '../../RAG files'
$ bash scripts/run_backend.sh # → http://localhost:8010
$ bash scripts/run_streamlit.sh # → http://localhost:8502
```

12.2 Cloud Deployment

CreditSense v2 is designed for zero-cost cloud deployment using the following topology:

Component	Platform	Notes
Streamlit UI	Streamlit Cloud	Frontend host — connects to backend via BACKEND_API_BASE_URL env
FastAPI API	Render.com	API service — Procfile + render.yaml included in repo
ChromaDB	Render Disk	Persistent disk mount for chroma_store directory
Groq LLM	Groq Cloud	API keys in environment; key rotation via groq_pool.py

12.3 Configuration Keys

Key	Default

CHROMA_PATH	./chroma_store
COLLECTION_NAME	creditsense_docs
RAG_TOP_K	8
DEFAULT_ANNUAL_INTEREST_RATE	0.15
BACKEND_API_BASE_URL	http://localhost:8010
BACKEND_CORS_ORIGINS	localhost:8501, localhost:8502
HINDI_FONT_PATH	path to NotoSansDevanagari .ttf
GROQ_API_KEY(S)	Groq Cloud API key(s) for LLM pool

13. Functional Requirements Status

The following table reflects the as-built status of all functional and non-functional requirements for Milestone 2:

Requirement	Status	Implementation Reference
Conversational borrower intake	■ Complete	agent/nodes.py, services/profile_parser.py
Safety / domain guardrails	■ Complete	services/guardrail.py
Form-based parameter seeding	■ Complete	api.py, app.py
Financial ratio computation	■ Complete	services/borrower_metrics.py
Heuristic risk scorer	■ Complete	services/risk_scorer.py
Deterministic underwriting	■ Complete	services/policy_engine.py
RAG retrieval (ONNX)	■ Complete	scripts/ingest.py, services/retriever.py
English report generation	■ Complete	services/report_generator.py
Hindi report generation	■ Complete	services/translator.py
English/Hindi PDF download	■ Complete	services/pdf_exporter.py
Public REST API	■ Complete	api.py (5 endpoints)
Deployment manifests	■ Complete	Procfile, render.yaml, runtime.txt
Automated CI/CD testing	■ Pending	Not yet in milestone folder
Auth / rate-limited API	■ Pending	Not yet implemented
Retrieval reranker	■ Pending	Top-k similarity only (no cross-encoder)

14. Gaps & Future Roadmap

Milestone 2 is functionally complete for local conversational underwriting. The following gaps and roadmap represent the path from current state to a production-hardened, enterprise-grade platform:

14.1 Known Gaps

Category	Gap	Risk
Authentication	No multi-user auth layer or backend token enforcement	Medium-High
Session Storage	No persistent conversation history across sessions	Medium
Retrieval Quality	Top-k similarity only — no reranker or hybrid lexical+dense	Medium
Testing	No pytest unit tests for parser/policy/retriever modules	Medium
Observability	No structured logging sink or health alerting dashboard	Medium
Rate Limiting	No per-user rate limiting middleware on API	Medium
Mobile UI	Streamlit UI not optimised for mobile workflows	Low
Corpus Refresh	Manual ingestion only — no automatic scheduler	Low

14.2 Three-Phase Roadmap

Phase 1 — Near-Term (1–2 weeks)

- Add pytest unit tests for borrower_metrics, policy_engine, profile_parser
- Implement CI pipeline for lint + tests + E2E scenario smoke test
- Add backend rate limiting and structured request logging middleware
- Add minimal auth (API key or JWT) for write/agent endpoints

Phase 2 — Mid-Term (2–6 weeks)

- Add retrieval reranking and citation confidence scoring stage
- Add structured trace visualisation endpoint and debug panel in UI
- Add persistent conversation storage with SQLite or Postgres
- Create dev/stage/prod environment profiles and release checklist

Phase 3 — Long-Term (6+ weeks)

- Implement policy versioning and explainability snapshots for auditors

- Build human-in-the-loop reviewer workflow for override escalations
- Add multilingual expansion beyond English and Hindi
- Add portfolio analytics dashboard and model monitoring suite
- Upgrade to clustered vector DB (Pinecone or Weaviate) for scale

15. Conclusion

CreditSense v2 successfully proves that modern Generative AI can be safely integrated into the highly-regulated fintech sector by combining flexible language chat with rigid, deterministic policy graphs. Through the collaborative efforts of the project team, the platform transitioned from a terminal prototype to a fully functional, cloud-deployed product yielding actionable bilingual credit reports.

Key Technical Achievements

Achievement	Impact
❖ Hybrid AI Architecture	Deterministic constraint envelope over LLM ensures zero policy hallucinations in final decisions.
❖ ONNX Migration	Reduced embedding stack from 2.2 GB (PyTorch) to 50 MB (ONNX) — enabling free-tier cloud deployment.
❖ LangGraph Determinism	6-node StateGraph enforces strict execution order; LLM cannot skip policy calculations.
❖ Bilingual Reports	Full English + Hindi PDF generation with Noto Devanagari font using fpdf2.
❖ RAG Grounding	Every report response is backed by cited RBI/NBFC regulatory corpus chunks.
❖ Full-Stack Deployment	Dual-stack (Streamlit + FastAPI) with Procfile and render.yaml for cloud deployment.

The architecture presented in this report — combining conversational AI, retrieval-augmented regulatory knowledge, deterministic underwriting rules, and bilingual accessibility — serves as a validated template for responsible AI adoption in financial services. Future milestones will harden this foundation with production-grade authentication, retrieval reranking, automated testing pipelines, and a human-in-the-loop reviewer workflow, progressively moving the system toward enterprise readiness.

This report was prepared for academic evaluation of the CreditSense v2 project. All architecture, implementation details, and diagrams reflect the as-built state of the system as of April 2026.